

Typical mode Serial Peripheral Interface (SPI) master controller based on APB with clock optimization

¹Vikram Keerthana, ²P Sridevi, ³D Rajitha

¹PG Scholar, ²Associate Professor, ³Associate Professor

^{1,2,3} Department of Electronics and Communication Engineering

^{1,2,3} Miracle Educational Society Group of Institutions, Bhogapuram-535216, AP, India

Abstract – A configurable SPI interface based on APB bus using four slaves is proposed in this thesis. The SPI interface transmission information is configured through APB bus, which can realize the master-slave mode switching, 4 kinds of clock transmission mode switching and MSB/LSB communication mode switching. Meanwhile the transmission rate can be selected and the transmission length can be variable from 1 to 32 bits. Based on the SPI transmission protocol, the proposed design uses a finite state machine method to control the transmission timing of the SPI interface. This design was verified through RTL simulation and Verilog test bench for verification. The verification results show that the functional of SPI interface is correct and the data transmission is stable.

Index Terms – SPI Interface, APB bus, clock polarity, clock phase.

I. INTRODUCTION

SPI interface connect to APB bus with Verilog HDL. This project implement an SPI module with an APB Bus interface. The project contains 4 modules: APB interface, SPI master, 4 SPI slave, and test bench. we connect the master and 4 spi slaves to each other. The data is send to the APB interface, the interface give it to the master. The master write and read the data from the SPI slaves at same time. During the connecting all 4 SPI transmission modes was performed, and each slave send's back their transmission mode The simultaneous receive and transmit. Programmable clock polarity and clock phase.

Programmable master clock frequency generation from 2 to 256. Fully synchronous design with one clock domain. AMBA 3 APB Slave interface. Supports APB slave transfer with and without wait state, APB SPI design is meant to be interfaced with slow-speed peripherals. The initial design will contain an APB slave on one side, which will initiate the transactions which can read data from and write data to SPI peripherals. Since SPI is a serial interface, in the case of a

write, the design will ensure that data obtained through the APB interface is completely transmitted on the SPI interface before it initiates a new transaction. In case of a read, the entire data will be collected from the SPI side and transmitted on the APB side, before the next transaction starts.

This paper is used to the design and implementation of an "APB-Based Serial Peripheral Interface (SPI) Core", a "versatile and efficient module" engineered for seamless, high-speed communication between a master device (such as a microcontroller) and various peripheral devices. Recognized for its simplicity and robust data transfer capabilities, this SPI Core is an "ideal solution for diverse applications", including embedded systems, sensor interfacing, and sophisticated communication modules.

II. RELATED WORK

The entire system is orchestrated by the "spi_top_module.v", which integrates all the core functionalities from its sub-blocks to provide a complete and operational SPI solution.

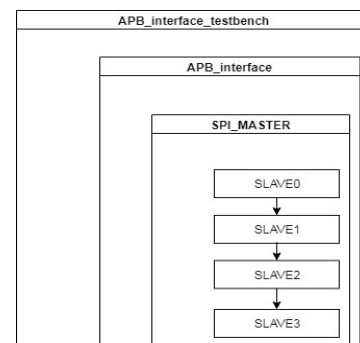


Figure 1: Project hierarchy

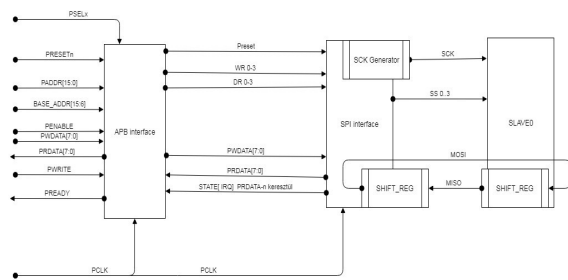


Figure 2: Overall system design

Overall system design of the project with the parameters defined in the modules.

A. Key features [3]

AMBA APB3 Compliance: Designed to integrate effortlessly into Advanced Peripheral Bus (APB) architectures.

Duplex Communication: Supports simultaneous two-way data transfer, enhancing communication efficiency.

Synchronous Data Transfer: Ensures reliable data transmission by synchronizing all operations with the clock signal.

Configurable Clock Characteristics: Provides control over clock polarity (CPOL) and clock phase (CPHA) to accommodate all four SPI modes (Mode 0, Mode 1, Mode 2, Mode 3).

Mode 0	CPOL=0, CPHA=0
Mode 1	CPOL=0, CPHA=1
Mode 2	CPOL=1, CPHA=0
Mode 3	CPOL=1, CPHA=1

High Clock Frequencies: Capable of operating at high clock speeds for rapid data transfer.

Interrupt Capabilities: Includes built-in mechanisms for efficient event handling and achieving low-latency data transfers.

Low Power Consumption: Optimised for power efficiency, making it suitable for battery-operated devices.

Flexible Data Handling: Supports both Least Significant Bit First Enable (LSBFE) and Most Significant Bit First transfer.

B. Architectural Components [4]

The SPI Core is modular, comprising several essential blocks, managed by the ``spi_top_module``, which work in concert to provide its full functionality.

``spi_apb_slave_select`` (APB Slave Interface):

1. This module is designed to interface with an APB Master and control the SPI device.
2. It manages read and write operations to the SPI's internal registers (control, status, data) via the APB bus.
3. Implements a state machine (IDLE, SETUP, ENABLE) to ensure proper APB protocol adherence.
4. Generates write and read enable signals and updates SPI control registers with appropriate masks.
5. Manages the SPI mode state machine (normal, wait, stop modes) and generates interrupt requests based on status flags.

``spi_baud_generator`` (Baud Rate Generator):

1. Responsible for generating the serial clock (SCLK) for the SPI device from the APB clock (PCLK).
2. It calculates the baud rate divisor using configurable SPI Baud Rate Pre-selection Bits (``sppr``) and SPI Baud Rate Selection Bits (``spr``).
3. The Baud Rate Divisor is calculated as $(SPPR+1) \times 2^{(SPR+1)}$.
4. Operates in different SPI modes (run, wait, stop) and adjusts clock generation accordingly.
5. Provides flags for sampling MISO and transmitting MOSI based on CPOL and CPHA.

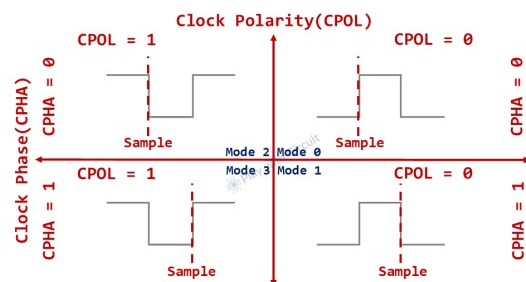


Figure 3: Timing diagram of sampling MISO and transmitting MOSI based on CPOL and CPHA

``spi_slave_select`` (SPI Slave Select Generator):

1. Manages the Slave Select (SS) signal for the SPI device when operating in master mode.
2. It ensures the SPI device is appropriately selected (SS low) and deselected (SS high) based on data transmission requirements and SPI mode settings.
3. Utilises a counter to keep SS low for a duration determined by the baud rate divisor.

Typical mode Serial Peripheral Interface (SPI).....

4. Generates the `receive\_data` signal to ensure proper data reception, asserted after SS has been low for the required period.
5. Provides a `tip` signal to indicate the transaction-in-progress status.

`spi\_shifter.v` (SPI Shifter):

1. Handles the core logic for serial-to-parallel and parallel-to-serial data conversion
2. Manages the Master Out Slave In (MOSI) and Master In Slave Out (MISO) data lines during SPI transactions.
3. Utilises configuration parameters such as clock polarity (CPOL), clock phase (CPHA), and least significant bit first enable (LSBFE) to correctly align data according to the SPI protocol settings.
4. Data to be transmitted via MOSI is loaded into a shift register when the `send\_data` signal is asserted and shifted out on the MOSI line.
5. Captures data from the MISO line into a temporary register when the `receive\_data` signal is asserted.

C. Register Address Map Overview [5]

The core provides access to key registers for configuration and status monitoring via the apb bus:

- Address 0: Spi control register 1 (rw): configures core operations like spi enable (spe), master/slave mode (mstr), clock polarity (cpol), clock phase (cpha), and slave select output enable (ssoe).
- Address 1: Spi control register 2 (rw): manages features such as mode fault detection enable (modfen), bidirectional output enable (bidiroe), and power conservation during wait mode (spiswai).
- Address 2: Spi baud rate register (rw): configures the serial clock frequency using preselection (sppr) and selection (spr) bits.
- Address 3: Spi status register (ro): provides read-only status flags, including spi transfer complete flag (spif), transmit buffer empty flag (sptef), and mode fault flag (modf).
- Address 4: Pi data register (rw) : used for temporary storage of data bytes to be transmitted or received.

III. PROPOSED METHOD

This project involves writing synthesizable RTL code (Verilog) for each component. The development process includes:

RTL Coding: Developing the Verilog code for each module. Validating the RTL coding style and identifying potential issues using linting processes. APB based SPI interface, transmission of information is done between a single master and four slaves. Selection is done based on chip select. Worked on Multi mode spi based on clock phase and clock polarity configuration between master and slaves

IV. RESULTS AND DISCUSSIONS

RTL level verification is completed and results are shown below:

The test application send 01010101 to the APB interface, the interface give it to the master. The master write and read the data from the SPI_slaves at same time. During the test all 4 spi transmission mode was performed, and each slave send's back their transmission mode 111100xx,

xx: 00 | 01 | 11 | 10

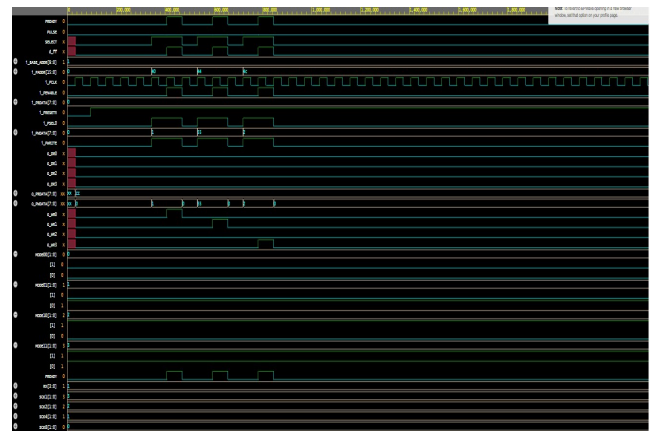


Figure 2: Simulated output waveform

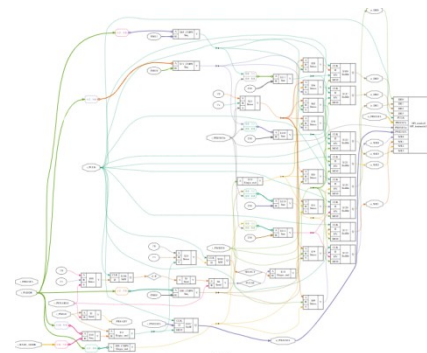


Figure 3: Synthesized output

V. CONCLUSION

SPI is implemented and verified in Verilog based on APB protocol with single master and four slaves in different modes of clock polarity and clock phase..

REFERENCES

- [1] Design and Verification of SPI Protocol using Verilog by Dr. K. Vijayalakshmi and Dr. G. Srihari: Focuses on the architectural specification, clock synchronization, master-slave logic, and simulation testbenches using HDL hardware design. [1]
- [2] Serial Port Complete [by Jan Axelson]: A classic engineering hardware reference providing code examples, bus layouts, and voltage level translation strategies for SPI and parallel interfaces.
- [3] Making Embedded Systems by Elecia White: Explains how to program synchronous serial communication directly from a software engineer's

Funding Declaration

The authors declare that no funds, grants, or other forms of financial support were received from any organization or institution for the conduct of this research.